

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Методы защиты информации»

ОТЧЕТ
к лабораторной работе № 1
на тему «Симметричная криптография. Стандарт шифрования ГОСТ
28147-89»

Выполнил

П. Д. Щиров

Проверил

Е. А. Лещенко

Минск 2024

СОДЕРЖАНИЕ

1 Постановка задачи.....	3
2 Краткие теоретические сведения.....	4
3 Результаты выполнения лабораторной работы.....	5
Выводы	6
Список использованных источников	7
Приложение А (обязательное) Листинг исходного кода	8

1 ПОСТАНОВКА ЗАДАЧИ

Целью выполнения данной лабораторной работы является реализация программных средств шифрования и дешифрования текстовых файлов при помощи стандарта шифрования ГОСТ 28147-89 в режиме простой замены.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

ГОСТ 28147-89 «Системы обработки информации. Защита криптографическая. Алгоритм криптографического преобразования» - устаревший государственный стандарт СССР, описывающий алгоритм симметричного блочного шифрования и режимы его работы. Симметричные криптосистемы – способ шифрования, в котором для шифрования и расшифрования применяется один и тот же криптографический ключ. Блочный шифр – разновидность симметричного шифра, оперирующего группам бит фиксированной длины – блоками, характерный размер которых меняется в пределах 64-256 бит. Если исходный текст меньше размера блока, перед шифрованием его дополняют.

ГОСТ 28147-89 является примером DES-подобных криптосистем, созданных по классической итерационной схеме Фейстеля. DES – алгоритм для симметричного шифрования, разработанный фирмой IBM и утвержденный правительством США в 1977 году как официальный стандарт. Размер блока для DES равен 64 битам. В основе алгоритма лежит сеть Фейстеля с 16 циклами и ключом, имеющим длины 56 бит.

Сеть Фейстеля, или конструкция Фейстеля – один из методов построения блочных шифров. Сеть состоит из ячеек, называемых ячейками Фейстеля. На вход каждой ячейки поступают данные и ключ. На выходе каждой ячейки получают измененные данные и измененный ключ. Все ячейки однотипны, и считается, что сеть представляет собой определенную итерированную структуру. Ключ выбирается в зависимости от алгоритма шифрования или дешифрования и меняется при переходе от одной ячейки к другой. При шифровании и расшифровании выполняются одни и те же операции, отличается только порядок ключей.

Выделяют четыре режима работы ГОСТ 28147-89:

- простой замены;
- гаммирование;
- гаммирование с обратной связью;
- режим выработки имитовставки.

Шифрование по ГОСТ 28147-89 в режиме простой замены — это один из режимов работы блочного симметричного шифра, который описан в российском стандарте ГОСТ 28147-89. Этот режим называют также электронной книгой кодов или режимом простой замены. В этом режиме каждый блок данных шифруется отдельно с использованием одного и того же ключа.

3 РЕЗУЛЬТАТЫ ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

В ходе выполнения лабораторной было реализовано программное средство шифрования и дешифрования текстовых файлов при помощи стандарта шифрования ГОСТ 28147-89 в режиме простой замены.

Начальный текст находится в файле input.txt. После шифрования зашифрованная информация помещается в файл encrypted.bin, после чего она снова дешифруется и помещается в файл decrypted.txt. Результат работы программы с исходными данными “qwertyui” представлен на рисунке 1.

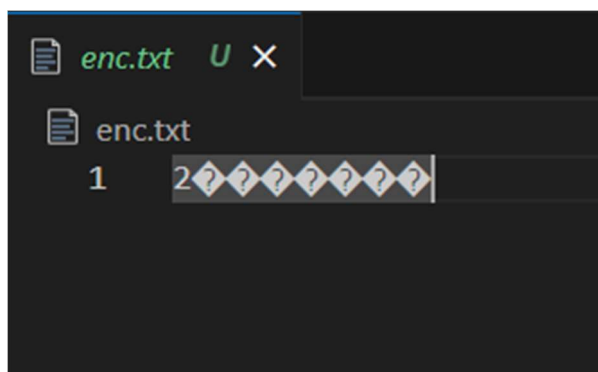


Рисунок 1 – Результат работы программы

Таким образом, в ходе данной лабораторной работы было реализовано программное средство шифрования и дешифрования текстовых файлов при помощи стандарта шифрования ГОСТ 28147-89 в режиме простой замены.

ВЫВОДЫ

В ходе данной лабораторной работы было реализовано программное средство шифрования и дешифрования текстовых файлов при помощи стандарта шифрования ГОСТ 28147-89 в режиме простой замены.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Алгоритм шифрования ГОСТ 28147-89 [Электронный ресурс]. – Режим доступа: <https://kaf401.rloc.ru/Criptfiles/gost28147/GOST28147.htm>. – Дата доступа: 02.09.2024.

[2] О шифровании ГОСТ 28147-89 [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/256843/>. – Дата доступа: 02.09.2024.

[3] Реализация алгоритма шифрования простой замены [Электронный ресурс]. – Режим доступа: <https://www.cyberforum.ru/thread1109400.html>. – Дата доступа: 02.09.2024.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг исходного кода

Листинг 1 – Исходный код

```
package main

import (
    "encoding/binary"
    "errors"
    "fmt"
    "os"
)

var sBoxes = [8][16]uint8{
    {4, 10, 9, 2, 13, 8, 0, 14, 6, 11, 1, 12, 7, 15, 5, 3},
    {14, 11, 4, 12, 6, 13, 15, 10, 2, 3, 8, 1, 0, 7, 5, 9},
    {5, 8, 1, 13, 10, 3, 4, 2, 14, 15, 12, 7, 6, 0, 9, 11},
    {7, 13, 10, 1, 0, 8, 9, 15, 14, 4, 6, 12, 11, 2, 5, 3},
    {6, 12, 7, 1, 5, 15, 13, 8, 4, 10, 9, 14, 0, 3, 11, 2},
    {4, 11, 10, 0, 7, 2, 1, 13, 3, 6, 8, 5, 9, 12, 15, 14},
    {13, 11, 4, 1, 3, 15, 5, 9, 0, 10, 14, 7, 6, 8, 2, 12},
    {1, 15, 13, 0, 5, 7, 10, 4, 9, 2, 3, 14, 6, 11, 8, 12},
}

func f(block uint32, key uint32) uint32 {
    sum := block + key
    for i := 0; i < 8; i++ {
        sum = (sum &^ (15 << (4 * i))) |
        (uint32(sBoxes[i][(sum>>(4*i))&15]) << (4 * i))
    }
    return (sum << 11) | (sum >> 21)
}

func encryptBlock(block *[2]uint32, key []uint32) {
    a, b := block[0], block[1]
    for i := 0; i < 32; i++ {
```

```

        t := b ^ f(a, key[i%8])
        b = a
        a = t
    }
    block[0], block[1] = b, a
}

func decryptBlock(block *[2]uint32, key []uint32) {
    a, b := block[0], block[1]
    for i := 31; i >= 0; i-- {
        t := b ^ f(a, key[i%8])
        b = a
        a = t
    }
    block[0], block[1] = b, a
}

func SimpleReplacementModeEnc(data []byte, key []uint32) error {
    if len(data)%8 != 0 {
        return errors.New("invalid data length")
    }

    temp := make([]byte, 8)
    for i := 0; i < len(data); i += 8 {
        block := [2]uint32{binary.LittleEndian.Uint32(data[i : i+4]),
        binary.LittleEndian.Uint32(data[i+4 : i+8])}

        encryptBlock(&block, key)

        binary.LittleEndian.PutUint32(temp[0:4], block[0])
        binary.LittleEndian.PutUint32(temp[4:8], block[1])
        copy(data[i:i+8], temp)
    }

    return nil
}

func SimpleReplacementModeDec(data []byte, key []uint32) error {

```

```

    if len(data)%8 != 0 {
        return errors.New("invalid data length")
    }

    temp := make([]byte, 8)
    for i := 0; i < len(data); i += 8 {
        block := [2]uint32{binary.LittleEndian.Uint32(data[i : i+4]),
        binary.LittleEndian.Uint32(data[i+4 : i+8])}
        decryptBlock(&block, key)
        binary.LittleEndian.PutUint32(temp[0:4], block[0])
        binary.LittleEndian.PutUint32(temp[4:8], block[1])
        copy(data[i:i+8], temp)
    }

    return nil
}

func main() {
    var err error
    defer func() {
        if err != nil {
            fmt.Println(err.Error())
        }
    }()

    key := []uint32{
        1234, 2345, 3456, 4567,
        5678, 6789, 7890, 8901,
    }

    inputFilename := "input.txt"
    encryptedFilename := "enc.txt"
    decryptedFilename := "dec.txt"

    data, err := os.ReadFile(inputFilename)
    if err != nil {

```

```
        return
    }

    err = SimpleReplacementModeEnc(data, key)
    if err != nil {
        return
    }

    err = os.WriteFile(encryptedFilename, data, 0644)
    if err != nil {
        return
    }

    err = SimpleReplacementModeDec(data, key)

    err = os.WriteFile(decryptedFilename, data, 0644)
}
```